

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Структурная и функциональная организация вычислительных
машин

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
на тему
«Стековое запоминающее устройство»

Выполнил
студент гр. 250541

Власов Р.Е.

Проверил
доцент, к.т.н. каф. ЭВМ

А.А. Воронов

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы	3
2 Исходные данные к работе.....	3
3 Теоретические сведения	4
4 Выполнение работы	6
5 Вывод.....	18

1 Цель работы

Целью лабораторной работы является разработка и проверка стекового запоминающего устройства для индивидуального варианта микро-ЭВМ. Устройство должно обеспечивать запись слова в стек, чтение последнего записанного слова, занесение результата арифметико-логического устройства, а также формирование признаков пустого и полного состояния.

Вариант 4 задает стек объемом семь позиций, направление роста вверх и правило, при котором указатель стека показывает последнюю занятую ячейку. Поэтому особое внимание уделяется не только передаче данных, но и правильному изменению указателя при переходах между пустым, частично заполненным и полностью заполненным состояниями.

В ходе выполнения работы необходимо разработать структурную схему устройства, определить входные и выходные сигналы, описать алгоритм работы, выполнить синтез проекта в Quartus II 9.1 и подтвердить корректность функционирования временными диаграммами. Проверка должна охватывать как обычные операции PUSH и POP, так и граничные режимы: запись в полный стек и чтение из пустого стека.

Таблица 1 – Этапы выполнения лабораторной работы

Этап	Содержание работы
1	Изучение индивидуального варианта, определение объема стека, направления роста и смысла указателя.
2	Разработка общей структуры устройства: блок управления, выбор источника данных, стековый массив, блок признаков состояния.
3	Определение входных и выходных сигналов, необходимых для подключения стека к тракту данных микро-ЭВМ.
4	Подготовка проекта Quartus II 9.1, выполнение компиляции и анализ полученной схемы соединения блоков.
5	Снятие временных диаграмм для операций PUSH, POP, PUSH_ALU, overflow и underflow.

2 Исходные данные к работе

Лабораторная работа выполняется для варианта 4. По заданию объем стека равен семи позициям, стек растет вверх, а указатель стека указывает на последнюю занятую ячейку. В практической схеме одна стековая позиция соответствует 16-разрядному слову тракта данных, поэтому устройство содержит семь 16-разрядных ячеек.

В качестве специального состояния пустого стека используется значение указателя, находящееся вне диапазона физических ячеек. Диапазон занятых ячеек задается значениями 0...6, а значение 7 означает, что в стеке нет данных. После первой успешной записи указатель принимает значение 0, после последующих записей увеличивается, а при чтении уменьшается.

Стековое устройство должно поддерживать три активных режима: запись данных из регистрового файла, чтение данных из стека и запись результата арифметико-логического устройства. Нулевой код команды используется как режим без операции, когда состояние стека сохраняется.

Таблица 2 – Исходные данные

Параметр	Значение
Номер варианта	4
Объем стека по заданию	7 позиций
Разрядность одной позиции в проекте	16 бит
Направление роста	Вверх, от младших номеров ячеек к старшим
Смысл указателя стека	Указывает на последнюю занятую ячейку
Пустой стек	SP=7 как специальное состояние вне диапазона 0...6
Полный стек	SP=6, заняты все семь ячеек

Для проверки используются коды команд 00, 01, 10 и 11. Код 00 соответствует режиму ожидания, код 01 выполняет запись данных из регистрового файла, код 10 выполняет чтение, а код 11 записывает в стек результат арифметико-логического устройства. Такая кодировка позволяет одной двухразрядной шиной управлять всеми проверяемыми режимами.

3 Теоретические сведения

Стек представляет собой запоминающее устройство с дисциплиной доступа LIFO. Это означает, что слово, записанное последним, будет считано первым. Такая организация применяется при сохранении адресов возврата, временных данных, промежуточных результатов вычислений и при выполнении вложенных процедур.

В отличие от произвольной памяти, стек не требует передачи адреса каждой операции извне. Адрес вершины хранится внутри устройства в виде

указателя стека. При записи и чтении указатель изменяется автоматически, поэтому внешнему блоку управления достаточно передать команду и данные.

Для варианта 4 указатель показывает последнюю занятую ячейку. Если стек пуст, последней занятой ячейки не существует, поэтому используется отдельное значение $SP=7$. Это значение удобно тем, что оно находится сразу за диапазоном реальных ячеек $0...6$ и позволяет однозначно отличить пустой стек от заполненного.

Операция PUSH выполняет запись нового слова. Если стек пуст, слово помещается в ячейку 0, а указатель принимает значение 0. Если в стеке уже есть данные, новая запись выполняется в следующую старшую ячейку, после чего указатель увеличивается на единицу.

Операция POP выполняет чтение слова из вершины. На выход передается содержимое ячейки, на которую указывает SP, затем указатель уменьшается. Если считывается ячейка 0, после операции стек становится пустым, и указатель возвращается в специальное состояние 7.

Контроль границ необходим для защиты данных и правильной диагностики ошибок. При попытке выполнить PUSH, когда указатель уже равен 6, новых свободных ячеек нет. В этом случае запись запрещается, состояние стека не изменяется, а на выходе ошибки формируется признак переполнения.

Аналогично, при попытке выполнить POP из пустого стека считывать нечего. Устройство не должно выдавать новое корректное слово и не должно изменять указатель. Вместо этого формируется признак чтения из пустого стека.

Режим PUSH_ALU отличается от обычного PUSH только источником данных. Вместо слова из регистрового файла на вход стекового массива подается результат арифметико-логического устройства. Это позволяет сохранять промежуточные результаты вычислений без дополнительной пересылки через общий регистр.

Флаги empty и full являются постоянными признаками состояния. Empty устанавливается, когда SP равен 7, а full устанавливается, когда SP равен 6. Ошибки overflow и underflow имеют импульсный характер: они появляются только в такте неправильной операции и затем сбрасываются.

Таблица 3 – Основные операции и признаки стекового устройства

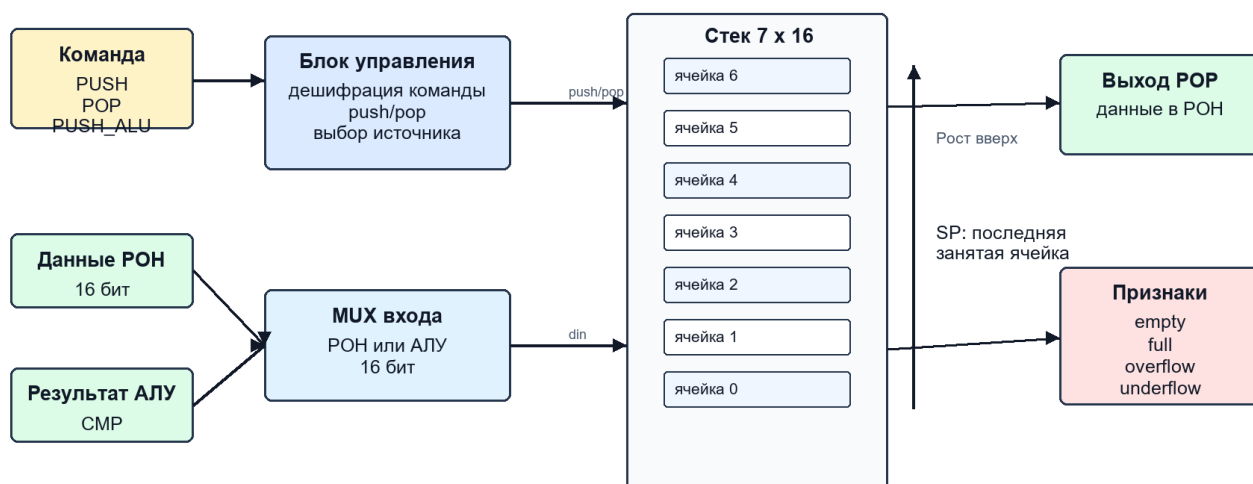
Понятие	Назначение
PUSH	Запись нового слова в стек при наличии свободной ячейки.
POP	Чтение слова из последней занятой ячейки с последующим изменением указателя.
PUSH_ALU	Запись результата арифметико-логического устройства в стек.
empty	Постоянный признак пустого стека, устанавливается при SP=7.
full	Постоянный признак полного стека, устанавливается при SP=6.
overflow	Ошибка записи в полный стек; состояние памяти и указателя не изменяется.
underflow	Ошибка чтения из пустого стека; состояние памяти и указателя не изменяется.

4 Выполнение работы

4.1 Структура разрабатываемого устройства

Структура стекового устройства выбрана блочной. На верхнем уровне выделяются блок управления командами, мультиплексор выбора источника данных, стековый массив и формирователь признаков состояния. Такое построение упрощает проверку, так как движение данных и изменение управляющих сигналов можно проследить независимо.

Вход команды определяет, должна ли выполняться запись, чтение или запись результата арифметико-логического устройства. При обычной записи источником служит вход `data_from_register_file_i`, а при режиме PUSH_ALU выбирается вход `alu_result_for_push_i`. Выход `data_popped_to_register_file_o` используется для передачи считанного слова обратно в регистровый файл.



Пустой стек: SP=7. После первой записи SP=0. Полный стек: SP=6.

Рисунок 4.1 – Обобщенная структура стекового запоминающего устройства

На схеме верхнего уровня показаны все основные внешние точки подключения: тактовый сигнал, сброс, код команды, входы данных, выход чтения, значение указателя и признаки состояния. Наличие отдельных диагностических выходов разрешения записи и чтения облегчает проверку временной диаграммы, поскольку по ним видно, как двухразрядная команда преобразуется во внутренние управляющие сигналы.

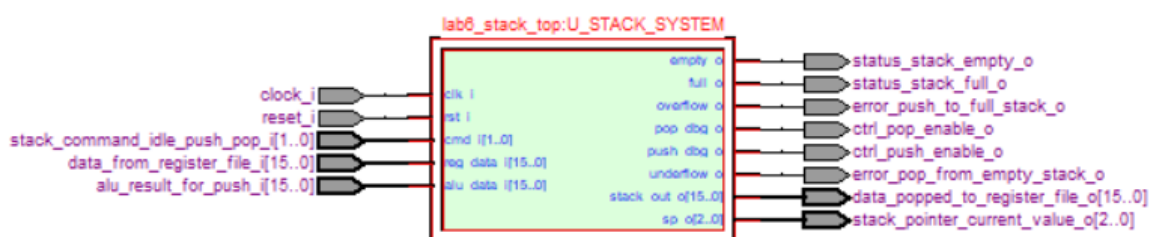


Рисунок 4.2 – Схема верхнего уровня стекового устройства с входными и выходными сигналами

4.2 Входные и выходные сигналы

Для проверки устройства используются сигналы, отражающие как информационную часть, так и управляющее состояние. Вход `clock_i` задает моменты изменения регистров, `reset_i` переводит стек в исходное пустое состояние, а `stack_command_idle_push_pop_i` выбирает выполняемую операцию.

Информационные входы разделены по назначению. data_from_register_file_i применяется для обычной записи из регистрового файла, а alu_result_for_push_i используется для записи результата арифметико-логического устройства. На выход data_popped_to_register_file_o поступает слово, считанное из вершины стека при операции POP.

Таблица 4 – Входные и выходные сигналы стекового устройства

Сигнал	Направление	Разрядность	Назначение
clock_i	in	1	Тактовый сигнал синхронной части устройства.
reset_i	in	1	Сброс стекового массива, выходного регистра, указателя и признаков ошибок.
stack_command_i dle_push_pop_i	in	2	Код операции: 00 - ожидание, 01 - PUSH, 10 - POP, 11 - PUSH_ALU.
data_from_register_file_i	in	16	Данные из регистрового файла для обычной записи в стек.
alu_result_for_push_i	in	16	Результат арифметико-логического устройства для режима PUSH_ALU.
data_popped_to_register_file_o	out	16	Слово, считанное из последней занятой ячейки стека.
stack_pointer_current_value_o	out	3	Текущее значение указателя стека SP.
status_stack_empty_o	out	1	Признак пустого стека.
status_stack_full_o	out	1	Признак полного стека.
error_push_to_full_stack_o	out	1	Признак попытки записи в полный стек.
error_pop_from_empty_stack_o	out	1	Признак попытки чтения из пустого стека.
ctrl_push_enable_o	out	1	Диагностический сигнал внутреннего разрешения записи.
ctrl_pop_enable_o	out	1	Диагностический сигнал внутреннего разрешения чтения.

4.3 Алгоритм работы

Алгоритм работы строится вокруг значения указателя SP. После сброса стек очищается, выход чтения принимает нулевое значение, флаги ошибок сбрасываются, SP устанавливается в 7, а признак empty становится равным единице. Это исходное состояние означает, что в стеке нет ни одного корректного слова.

При поступлении команды PUSH устройство сначала проверяет, не достигнут ли верхний предел. Если стек уже полон, запись не выполняется и формируется overflow. Если свободное место есть, новое слово записывается в следующую ячейку, а указатель переводится на эту ячейку. Для пустого стека первой занятой ячейкой становится ячейка 0.

При поступлении команды POP устройство проверяет, не пуст ли стек. Если стек пуст, формируется underflow и состояние не меняется. Если стек содержит данные, на выход передается слово из ячейки SP, после чего указатель уменьшается. Когда считывается последняя оставшаяся ячейка 0, следующей величиной указателя становится 7.

Команда PUSH_ALU использует тот же механизм записи, что и обычная команда PUSH, но перед стековым массивом выбирается другой источник входного слова. Это позволяет показать, что стековое устройство подключается не только к регистровому файлу, но и к результату вычислительного блока.

Таблица 5 – Алгоритм работы стекового устройства

Шаг	Действие	Результат
1	Подается reset_i=1.	SP=7, стек пуст, empty=1, full=0, ошибки сброшены.
2	Поступает команда PUSH при наличии свободной ячейки.	В стек записывается входное слово, SP указывает на новую последнюю занятую ячейку.
3	Поступает команда POP при непустом стеке.	На выход передается последнее записанное слово, SP смещается к предыдущей занятой ячейке.
4	Поступает команда PUSH_ALU.	В стек записывается слово с входа результата арифметико-логического устройства.
5	Выполняется PUSH при полном стеке.	Запись запрещается, SP не меняется, устанавливается overflow.

Шаг	Действие	Результат
6	Выполняется POP при пустом стеке.	Чтение запрещается, SP не меняется, устанавливается underflow.

ЛР6. Контрольная временная диаграмма стека

	t0	t1	t2	t3	t4	t5	t6	t7	t8
Режим	RST	PUSH	PUSH	POP	PUSH_ALU	FILL	PUSH	POPx7	POP
DIN	0000	1111	2222	----	000F	3333..7777	8888	----	----
DOUT	0000	0000	0000	2222	2222	2222	2222	1111	1111
SP	7	0	1	0	1	6	6	7	7
EMPTY	1	0	0	0	0	0	0	1	1
FULL	0	0	0	0	0	1	1	0	0
OVER	0	0	0	0	0	0	1	0	0
UNDER	0	0	0	0	0	0	0	0	1

SP=7 означает пустой стек. При росте вверх первая запись занимает ячейку 0, полный стек соответствует SP=6.

Рисунок 4.3 – Сводная таблица контрольных операций стекового устройства

4.4 Реализация устройства

Внутренняя структура состоит из блока управления и собственно стекового массива. Блок управления дешифрирует двухразрядную команду и формирует сигналы записи, чтения и выбора источника данных. Стековый массив содержит семь 16-разрядных ячеек, регистр указателя, выходной регистр и логику контроля границ.

Перед входом стекового массива расположен мультиплексор источника данных. Если выполняется обычный PUSH, на вход массива поступает слово из регистрового файла. Если выполняется PUSH_ALU, на вход массива передается результат арифметико-логического устройства. Благодаря этому оба режима записи используют общий стековый массив и одинаковые правила изменения указателя.

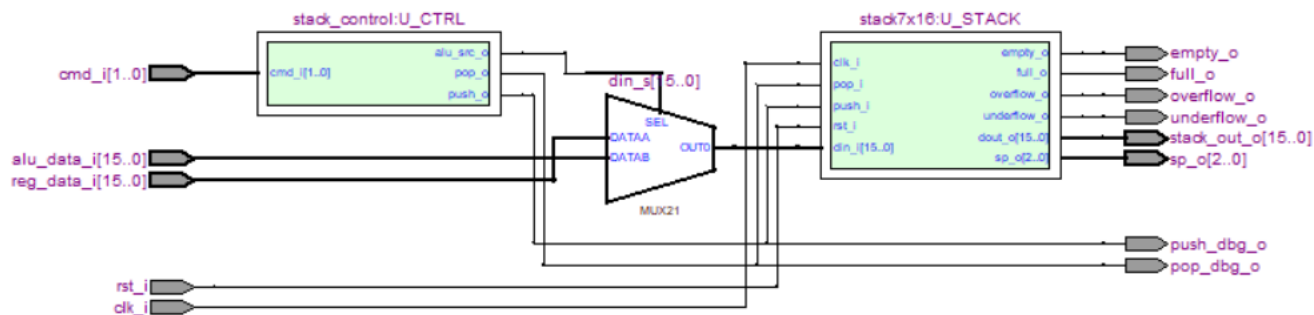


Рисунок 4.4 – Внутренняя структура устройства: блок управления, выбор источника данных и стековый массив

Блок дешифрации команды формирует два основных управляющих сигнала. Для кода 01 активируется запись, для кода 10 активируется чтение, а для кода 11 активируется запись и одновременно выбирается источник данных от арифметико-логического устройства. В режиме 00 управляющие сигналы неактивны, поэтому состояние стека сохраняется.

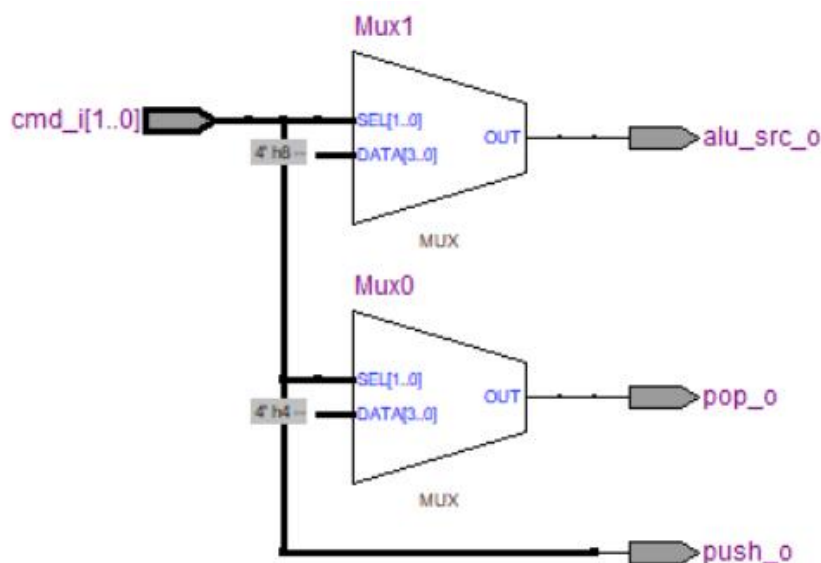


Рисунок 4.5 – Блок дешифрации команд PUSH, POP и PUSH_ALU

Стековый массив выполняет основные операции по фронту тактового сигнала. На больших схемах показаны регистры ячеек, цепи выбора адреса, выходной регистр и логика формирования признаков. Эти изображения вынесены на горизонтальные страницы, чтобы сохранить читаемость соединений и подписей сигналов.

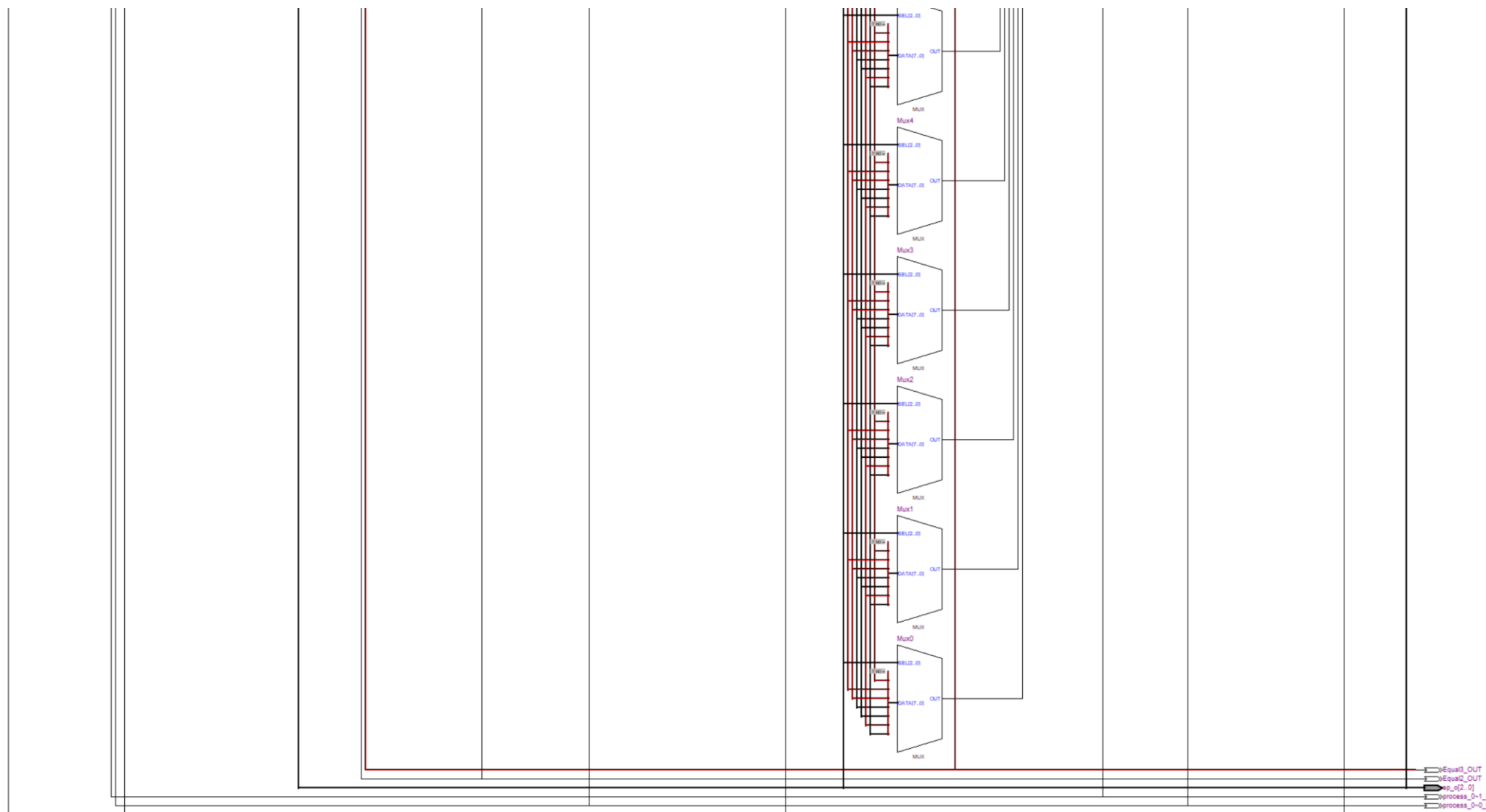


Рисунок 4.7 – Схема стекового массива и контроля указателя, часть 2

Контроль границ выполняется по текущему значению SP. При SP=7 устройство считается пустым, поэтому команда POP приводит к underflow. При SP=6 устройство считается полным, поэтому команда PUSH приводит к overflow. В обоих ошибочных случаях состояние указателя и содержимое памяти не должны изменяться.

Таблица 6 – Назначение функциональных блоков

Блок	Назначение
Блок управления командами	Преобразует код 00/01/10/11 в сигналы ожидания, записи, чтения и выбора источника данных.
Мультиплексор входных данных	Выбирает слово из регистрового файла или результат арифметико-логического устройства.
Стековый массив	Хранит семь 16-разрядных слов и выдает слово из последней занятой ячейки.
Регистр указателя SP	Хранит номер последней занятой ячейки либо специальное состояние 7 для пустого стека.
Блок признаков состояния	Формирует empty, full, overflow и underflow по состоянию стека и выполняемой операции.
Выходной регистр	Сохраняет слово, считанное при операции POP, для передачи в регистровый файл.

4.5 Функциональное моделирование

Функциональная проверка выполнена по последовательности операций, которая охватывает рабочие и граничные режимы. Сначала выполняется сброс, затем две операции записи, чтение последнего записанного слова, запись результата арифметико-логического устройства, заполнение стека до полного состояния, попытка записи в полный стек и попытка чтения из пустого стека.

На временной диаграмме особое внимание уделяется указателю SP. После сброса он равен 7. После первой записи он становится 0, после второй записи - 1. При чтении слова 2222h указатель возвращается к 0, затем после PUSH_ALU снова становится 1. Заполнение оставшимися словами доводит SP до 6, что соответствует полному стеку.

Ошибочные ситуации проверяются отдельно. При попытке записать слово 8888h в полный стек указатель остается равным 6, а сигнал error_push_to_full_stack_o устанавливается на один такт. После последовательного чтения всех семи слов указатель возвращается в 7, empty

становится равным единице, и следующая команда POP вызывает error_pop_from_empty_stack_o.

Таблица 7 – Контрольные точки временной диаграммы

Проверка	Условие	Ожидаемый результат
Сброс	reset_i=1	SP=7, empty=1, full=0, ошибки отсутствуют.
PUSH 1111h	cmd=01, data_from_register_file_i=1111h	Занята ячейка 0, SP=0.
PUSH 2222h	cmd=01, data_from_register_file_i=2222h	Занята ячейка 1, SP=1.
POP	cmd=10	На выходе появляется 2222h, SP=0.
PUSH_ALU	cmd=11, alu_result_for_push_i=000Fh	В стек занесено 000Fh, SP=1.
Заполнение	Последовательные записи 3333h...7777h	SP=6, full=1.
Overflow	PUSH 8888h при полном стеке	Формируется ошибка overflow, SP остается 6.
Underflow	POP при SP=7	Формируется ошибка underflow, SP остается 7.

Полученная временная диаграмма подтверждает дисциплину LIFO: после записи 1111h и 2222h первой считывается величина 2222h. Это показывает, что вершина стека задается последней занятой ячейкой, а операция POP обращается именно к этой ячейке.

Проверка PUSH_ALU подтверждает правильность выбора источника данных. Несмотря на то что вход регистрового файла сохраняет предыдущее значение, в стек помещается слово 000Fh с отдельного входа результата арифметико-логического устройства. Это соответствует требуемому режиму занесения результата вычисления в стек.

4.6 Результаты синтеза

После подготовки проекта выполнена полная компиляция в Quartus II 9.1. Ошибки компиляции отсутствуют, что подтверждает корректность соединения блоков, согласование разрядностей входных и выходных шин, а также правильное формирование сигналов состояния.

Основная часть аппаратных ресурсов расходуется на регистровый массив из семи 16-разрядных слов, регистр выходных данных и регистр указателя. Комбинационные элементы используются для дешифрации команд, выбора источника данных, изменения указателя и формирования признаков empty, full, overflow и underflow.

Таблица 8 – Результаты синтеза устройства

Показатель	Значение
Combinational ALUTs	68
Dedicated logic registers	135
Pins	61
Block memory bits	0
Результат компиляции	Full compilation: 0 errors

5 Вывод

В ходе лабораторной работы разработано стековое запоминающее устройство для варианта 4. Устройство содержит семь стековых позиций, работает с 16-разрядными словами, растет вверх и использует указатель, показывающий последнюю занятую ячейку.

Разработанная структура включает блок управления командами, выбор источника данных, стековый массив, регистр указателя, выходной регистр и блок признаков состояния. Такая организация обеспечивает выполнение

операций PUSH, POP и PUSH_ALU, а также контроль граничных состояний стека.

Функциональная проверка подтвердила корректность работы устройства. После сброса стек находится в пустом состоянии с SP=7; при записях указатель последовательно изменяется от 0 до 6; при чтении данные выдаются в порядке LIFO; при полном стеке формируется overflow, а при чтении из пустого стека формируется underflow.

Таким образом, стековое устройство соответствует исходным данным и может использоваться в составе разрабатываемой микро-ЭВМ для временного хранения операндов, результатов вычислений и служебной информации.